Contender 35: Transfinite Metalanguage Reflection over Tree Ordinals

A research proposal for the BusyLean project

13 October 2026

Abstract

The accepted BusyLean Contender 34 iterates a first-order language-closure operator along a countable tree ordinal. This report proposes a qualitatively stronger construction, Contender 35, by raising the closure mechanism itself one functional type. A new simply typed metalanguage has a primitive

$$\mathsf{lift} : (\mathsf{CTree} \to \mathsf{CTree}) \to \mathsf{CTree} \to \mathsf{CTree},$$

so programs can construct tree transformers, pass them to a lower-level language-closure operator, and continue computing with the resulting trees. The total semantic interpreter, raw syntax, type checker, and finite program enumerator are all defined constructively in Lean. The induced metalanguage closure is then iterated by structural recursion along Contender 34's own tree. Finally, BusyLean's family-aware address reflection and W-tree-ranked lift are applied with the full Contender 34 family as seed.

The proposed closed number is

$$C_{35} = F_{35}(F_{35}(42)),$$

where F_{35} is the new metalanguage-reflection family. The package contains Lean modules proving

$$C_{34} < C_{35},$$

without mentioning the constant `contender_34` in the dependency closure of the new definition. The strict numerical comparison is formal. The claim that the construction represents a major growth-rate jump is a separate, structural claim: Contender 35 closes over *operators that close whole languages*, rather than merely extending the ordinal height of the same first-order closure. No exact proof-theoretic ordinal is asserted.

Contents

1 Executive summary **4**

2 The incumbent and the design target **4**
2.1 What Contender 34 already does 4
2.2 Requirements for a genuine successor 5

3 The functional-type jump **6**
3.1 Two operator levels 6
3.2 Why the new primitive is first-class enough 6

4 Syntax, typing, and total semantics **7**
4.1 Reusing the existing type universe . . . 7
4.2 Semantic evaluation . . . 7
4.3 Raw terms and a decidable checker . . . 8
4.4 Finite enumeration without evaluation at build time . . . 8

5 The metalanguage closure **8**
5.1 Definition . . . 8
5.2 What one successor stage contains . . . 9
5.3 Recovery of lower-level behavior . . . 9

6 Transfinite metalanguage iteration **9**
6.1 Structural recursion on countable trees . . . 9
6.2 Self-height without circular definition . . . 10

7 From a tree to a natural number **10**
7.1 Address-ladder programs . . . 10
7.2 Family-aware reflection . . . 10
7.3 The seed, schedule, and rank . . . 11

8 Formal comparison with Contender 34 **11**
8.1 The proof chain . . . 11
8.2 Why this is not a disguised use of the previous constant . . . 12

9 Constructive computability **12**
9.1 Why the number exists as a computation . . . 12
9.2 No hidden nonconstructive shortcut . . . 13

10 Why the increase is significant **13**
10.1 Parameter growth versus type growth . . . 13
10.2 A schematic separation example . . . 13
10.3 Four amplifications are composed . . . 13
10.4 What is and is not proved . . . 14

11 Safety, predicativity, and consistency posture **14**
11.1 No self-interpreter . . . 14
11.2 Predicative simple types . . . 14
11.3 Fallback constructors . . . 14

12 Verification and audit protocol **14**
12.1 Exact environment . . . 14
12.2 Checks performed . . . 15
12.3 Verification status . . . 15

13 Integration into BusyLean **15**

14 Alternative designs considered **15**
14.1 Another tree-ordinal stacking round . . . 15
14.2 A second W-ranked lift only . . . 16
14.3 Untyped universal interpreters . . . 16
14.4 Impredicative language universes . . . 16

15 Further research program **16**
15.1 Formal embedding and rank theorems . . . 16
15.2 All finite meta-levels . . . 16
15.3 Tree-indexed functional levels . . . 16
15.4 A typed code universe with certified evaluation . . . 17
15.5 Exact proof-theoretic calibration . . . 17
15.6 Performance engineering . . . 17

16 Conclusion **17**

A Complete Lean source: the metalanguage **17**

B Complete Lean source: the number **21**

C Complete Lean source: comparison theorems **23**

D Artifact map **24**

1 Executive summary

The progression from BusyLean Contenders 32 through 34 can be read as a systematic escalation of the same idea:

1. make tree-ordinal constructions into programs;
2. make the whole program language into the next tree transformer;
3. iterate that language transformer along a countable tree ordinal.

Contender 35 takes the next natural step. It does not merely choose a larger height for the operator used by Contender 34. Instead it introduces an object language whose programs can invoke the *language-closure operator itself* on dynamically constructed tree transformers. This is a one-functional-type jump:

$$\underbrace{\mathsf{CTree} \to \mathsf{CTree}}_{\text{tree transformer}} \quad \longrightarrow \quad \underbrace{(\mathsf{CTree} \to \mathsf{CTree}) \to (\mathsf{CTree} \to \mathsf{CTree})}_{\text{language-closure operator}}.$$

The new closure operator is then iterated along a tree whose index is itself produced by Contender 34.

The contribution has three layers.

Typed metalanguage.
A total, predicative, simply typed calculus over natural numbers and countable trees, with primitives `boost` and `lift`. Evaluation is direct structural recursion into Lean values.

Transfinite closure.
A functional $\mathsf{MetaLang} : \mathcal{M} \to \mathcal{M}$, where $\mathcal{M} = ((\mathsf{CTree} \to \mathsf{CTree}) \to \mathsf{CTree} \to \mathsf{CTree})$, followed by structural iteration along countable trees.

Numeric reflection.
The resulting tree supplies the atoms for the already formalized System-F ladder normalization and address reflection. A W-tree-ranked lift is seeded with `family34 n = F34 (F34 n)`.

The formal comparison is intentionally simple once the stronger closure object has been built: the reflection transformer is progressive, so $F_{35}(n)$ strictly exceeds the entire seed family at every input; the two outer applications then give $C_{34} < C_{35}$.

2 The incumbent and the design target

2.1 What Contender 34 already does

Let

$$\mathcal{O} = \mathsf{CTree} \to \mathsf{CTree}$$

be the type of countable-tree transformers. The accepted construction defines `langLift` of type $\mathcal{O} \to \mathcal{O}$. Informally, $\mathsf{Lang}(b)(t)$ is the limit of $P(t)$ over all closed, well-typed programs $P : \mathsf{CTree} \to \mathsf{CTree}$ in a simply typed language whose distinguished primitive is the transformer b.

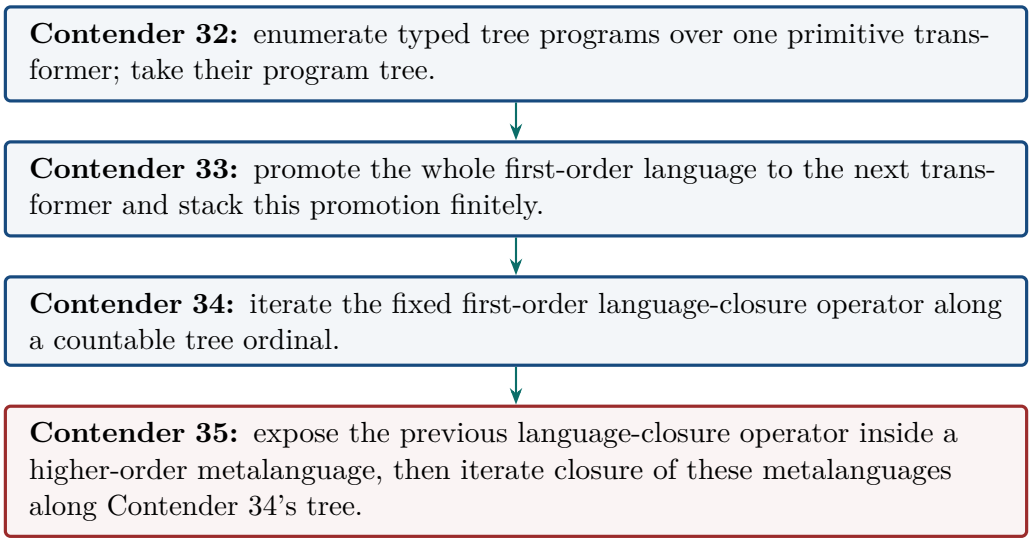


Figure 1: The proposed step changes the functional type of the closure rule, not only its iteration height.

Contender 33 iterates Lang a finite number of times. Contender 34 replaces that finite iteration count by a countable tree ordinal. In schematic form,

$$\begin{aligned} B_0 &= \mathsf{boost}_{31}, \\ B_{a+1} &= \mathsf{Lang}(B_a), \\ B_{\lim f}(t) &= \lim_i B_{f(i)}(t). \end{aligned}$$

At input n, the height and argument are both a tree inherited from Contender 33. The numerical layer then reflects programs over addresses in that tree and applies BusyLean's W-ranked hyperreflection.

This is already far beyond a hand-written ordinal notation. Nevertheless, there is a clear type-theoretic ceiling: throughout the construction, Lang is a fixed meta-level function. Object-language programs can apply the current tree transformer, but cannot receive a language-closure operator as data and apply it to a transformer they have computed.

2.2 Requirements for a genuine successor

A serious successor should satisfy all of the following.

R1. Constructive closed natural. The final term must be a closed Lean `Nat`, with no oracle, noncomputable choice, unsafe implementation, partial definition, or opaque stand-in.

R2. Formal strict comparison. A Lean theorem must prove `contender_34 < contender_35`.

R3. No constant smuggling. The definition of `contender_35` must not depend on the constant `contender_34`. It may use the previous construction as a family, as BusyLean's rules permit.

R4. Qualitative increase. The new machinery should increase the expressive type of the closure operation, not only enlarge a numeric parameter or repeat the exact same reflection one more time.

R5. Small trusted base. The extension should reuse the existing normalization, address well-foundedness, and W-ranked lift theorems rather than adding an unverified ordinal comparison or a new axiom.

R6. Kernel tractability. Definitions and comparison proofs must elaborate without evaluating the represented number or enumerating the huge program lists.

The construction below was chosen to meet these requirements simultaneously.

3 The functional-type jump

3.1 Two operator levels

Write

$$T = \mathsf{CTree}, \qquad \mathcal{O} = T \to T, \qquad \mathcal{M} = \mathcal{O} \to \mathcal{O}.$$

The first-order language closure has type

$$\mathsf{Lang} : \mathcal{M}.$$

Contender 34 iterates this particular element of $\mathcal{M}$ along a tree. The new idea is to define a closure operation on *arbitrary elements of* $\mathcal{M}$:

$$\mathsf{MetaLang} : \mathcal{M} \to \mathcal{M}.$$

Thus a successor step no longer says merely "close under all programs using this tree transformer." It says "close under all programs that can use both this tree transformer and the previous *closure operator*."

The distinction is the same structural distinction that separates a hierarchy of functions from a hierarchy of functionals acting on that hierarchy. It is not an exact ordinal analysis, but it is a robust reason to expect a major increase in definability and growth.

3.2 Why the new primitive is first-class enough

The metalanguage adds the constructor

```
| lift : Tm Gamma (.arr .tree .tree) ->
    Tm Gamma .tree -> Tm Gamma .tree
```

with semantic interpretation

```
| .lift b t, rho =>
    liftF (b.eval liftF boostF rho) (t.eval liftF boostF rho)
```

Although `lift` is presented as a primitive constructor, ordinary lambda abstraction makes the corresponding functional available as a value. For example, the term

$$\lambda b : T \to T.\ \lambda t : T.\ \mathsf{lift}\ b\ t$$

has type $(T \to T) \to T \to T$. Programs may pass such functionals to higher-order combinators, iterate them by natural recursion, and use them in tree recursion at arbitrary simple types.

The calculus remains predicative and simply typed. There is no type of all types, no impredicative self-encoding, and no general recursion. In particular, this proposal does not rely on a normalization conjecture: its interpreter is a structurally recursive Lean definition.

4 Syntax, typing, and total semantics

4.1 Reusing the existing type universe

The metalanguage reuses the three type constructors from the existing tree language:

$$A, B ::= \mathsf{nat} \mid \mathsf{tree} \mid A \to B.$$

Their denotations are Lean's `Nat`, the repository's `CTree`, and ordinary Lean function spaces. Variables are intrinsically typed de Bruijn indices. Consequently, every constructor of `MTL.Tm Gamma A` denotes a Lean value of the interpretation of A.

The term constructors are:

- variables, lambda abstraction, and application;
- zero, successor, and primitive recursion on naturals;
- zero, successor, and countable limit for trees;
- structural recursion over trees at any simple result type;
- `boost : tree -> tree`;
- `lift : (tree -> tree) -> tree -> tree`.

4.2 Semantic evaluation

For a closure operator $L \in \mathcal{M}$ and a base transformer $b \in \mathcal{O}$, the interpreter has type

$$\mathsf{eval}_{L,b} : \mathsf{Tm}(\Gamma, A) \to [\![\Gamma]\!] \to [\![A]\!].$$

Every recursive call is made on a strict subterm, except for calls to the already-total natural and tree iterators. The two distinguished cases are

$$[\![\mathsf{boost}\ t]\!]_{L,b} = b([\![t]\!]_{L,b}),$$
$$[\![\mathsf{lift}\ p\ t]\!]_{L,b} = L([\![p]\!]_{L,b})([\![t]\!]_{L,b}).$$

This makes totality transparent. A program cannot loop by applying `lift`; it can only invoke the total semantic function supplied as `liftF`.

Proposition 4.1 (Constructive evaluation)**.** *For every closed well-typed metalanguage term $p : A$, every total $L \in \mathcal{M}$, and every total $b \in \mathcal{O}$, the Lean expression $\mathsf{eval}_{L,b}(p)$ reduces in principle to a value of $[\![A]\!]$.*

Proof. The evaluator is accepted by Lean's structural termination checker. Natural and tree recursion are defined by structural recursion on their respective inductive inputs. Function values are ordinary total Lean functions. No classical choice or partial evaluator occurs. □

4.3 Raw terms and a decidable checker

The package also supplies an untyped raw syntax. Lambda nodes carry their argument type, so inference is syntax-directed. The checker returns either `none` or a dependent pair $\langle A, p\rangle$ with $p : \mathsf{Tm}(\Gamma, A)$. Equality of simple types is decidable. The new case verifies that the first child has type $T \to T$ and the second has type T:

```
| .lift b r => do
    let <_, tb> <- infer Gamma b
    let <_, tr> <- infer Gamma r
    let tb' <- cast? (.arr .tree .tree) tb
    let tr' <- cast? .tree tr
    pure <.tree, .lift tb' tr'>
```

Here and below, ASCII angle-bracket notation in displayed excerpts stands for Lean's dependent-pair brackets in the source file.

Because the output is intrinsically typed, checker soundness is built into its result type. There is no separate preservation theorem to trust at this layer.

4.4 Finite enumeration without evaluation at build time

`rawsUpTo n` is a finite list of raw terms with construction depth at most n. It extends the previous enumerator by the finite product

$$\{\mathsf{lift}(p, q) : p, q \in \mathsf{rawsUpTo}(n-1)\}.$$

Filtering this list through the checker and retaining closed terms of type $T \to T$ yields `MTL.funsUpTo`. The definition is computable but is not evaluated while Lean checks the module. The enormous list remains under lambda binders inside the tree representation.

5 The metalanguage closure

5.1 Definition

For $L \in \mathcal{M}$, $b \in \mathcal{O}$, and $t \in T$, define

$$\mathsf{MetaLang}(L)(b)(t) = \lim_{m\in\mathbb{N}} \lim_{i\in\mathbb{N}} P_{m,i}^{L,b}(t),$$

where $P_{m,i}^{L,b}$ is the i-th closed, checked metalanguage program of type $T \to T$ appearing in the finite enumeration of depth at most m, or the zero transformer when the index is out of range.

In Lean:

```
def metaLangLift
    (liftF : (CTree -> CTree) -> CTree -> CTree)
    (b : CTree -> CTree) (t : CTree) : CTree :=
  .lim fun m => .lim fun i =>
    (((MTL.funsUpTo liftF b m)[i]?).map fun p => p t).getD .zero
```

The result is again an element of $\mathcal{M}$ when the first argument is fixed:

$$\mathsf{MetaLang} : \mathcal{M} \to \mathcal{M}.$$

5.2 What one successor stage contains

A single metalanguage closure contains, as branches, values of programs that can perform all of the following before returning a tree:

1. use zero, successor, and arbitrary countable limits;
2. recurse along an input tree at any finite simple type;
3. compose, iterate, and higher-order manipulate the base transformer b;
4. construct a new transformer $q : T \to T$ inside the program;
5. call the previous closure operator $L(q)$ on a dynamically computed tree;
6. nest such calls under natural recursion, tree recursion, lambdas, and higher-order applications.

The fourth and fifth items are the crucial novelty. In Contender 34 the meta-level recursion decides in advance which transformer is promoted to the next language. Here a program can decide which transformer to promote as part of its computation.

5.3 Recovery of lower-level behavior

Ignoring the `lift` constructor gives a syntax-preserving embedding of the previous tree language into the metalanguage. Under this embedding, evaluation with base transformer b agrees with the previous evaluator. Therefore the new closure does not discard the first-order program universe; it contains it and adds programs that invoke L.

A fully formal syntactic embedding theorem would be a useful strengthening of the package, but it is not needed for the strict numerical comparison. The current tree deliberately includes Contender 34's tree as branch zero, while the numerical family is independently seeded with the whole Contender 34 family.

6 Transfinite metalanguage iteration

6.1 Structural recursion on countable trees

Define $M_a \in \mathcal{M}$ by structural recursion on a countable tree a:

$$\begin{aligned} M_0 &= \mathsf{Lang}, \\ M_{a+1} &= \mathsf{MetaLang}(M_a), \\ M_{\lim f}(b)(t) &= \lim_{i \in \mathbb{N}} M_{f(i)}(b)(t). \end{aligned}$$

The repository's auxiliary `supL` and `supH` constructors use the conservative base case Lang, matching the treatment in earlier tree language modules. Lean code:

```
def metaBoostT : CTree -> (CTree -> CTree) -> CTree -> CTree
  | .zero => langLift
  | .succ a => metaLangLift (metaBoostT a)
  | .lim f => fun b t => .lim fun i => metaBoostT (f i) b t
  | .supL _ => langLift
  | .supH _ => langLift
```

Termination is inherited directly from the inductive structure of `CTree`. No comparison of ordinal codes is required.

6.2 Self-height without circular definition

At input n, let

$$T_{34}(n) = \mathsf{treeStackedTree}(n).$$

The new tree is

$$T_{35}(n) = \lim_i \begin{cases} T_{34}(n), & i = 0, \\ M_{T_{34}(n)}(B_{T_{34}(n)})(T_{34}(n)), & i \neq 0, \end{cases}$$

where $B_a = \mathsf{boostT}(a)$ is the first-order hierarchy from Contender 34.

This is "self-height" only in the benign sense that a previously constructed finite Lean value $T_{34}(n)$ is used as an index and as an argument. There is no recursive equation in which $T_{35}(n)$ occurs on its own right-hand side. Hence the definition remains ordinary structural recursion.

The exact Lean definition is:

```
def metaLanguageTree (n : Nat) : CTree :=
  .lim fun i =>
    if i = 0 then treeStackedTree n
    else metaBoostT (treeStackedTree n)
      (boostT (treeStackedTree n)) (treeStackedTree n)
```

Design principle 6.1 (Height, base, and argument all advance)**.** A weak successor could keep the same base transformer and merely increase the iteration height. The present construction advances all three inputs: $T_{34}(n)$ is the meta-iteration height, indexes the first-order base hierarchy, and is the tree to which the resulting operator is applied.

7 From a tree to a natural number

7.1 Address-ladder programs

BusyLean already proves that strict descent through addresses of any `CTree` is well-founded. Applying its transfinite System-F ladder to $T_{35}(n)$ therefore gives a universe of atom types without a new ordinal notation theorem. The new module defines:

- `sn_erase_metatree`: every accepted erased term over these addresses is strongly normalizing;
- `metavalue`: normalize a checked program relative to a packed natural-number oracle and decode its numeral;
- `metatreeLadderUpTo n`: the finite list of candidate programs at depth at most n.

The value function is total because it carries the strong-normalization proof required by the existing normalizer. Ill-typed candidates return zero.

7.2 Family-aware reflection

For a family $f_i : \mathbb{N} \to \mathbb{N}$, define

$$R_{35}(\vec{f}, n) = n + 1 + \max\left(\max_{i \le n} f_i(n), \max_{p \in \mathcal{P}_n} \mathsf{metavalue}(n, \mathsf{pack}(\vec{f}), n, p)\right),$$

where $\mathcal{P}_n$ is the finite metalanguage-tree ladder. The added $n+1$ makes strictness immediate even when one of the maxima is attained.

Lean packages this as `reflectMetaFamily` and the corresponding `Transformer`. Two elementary inequalities drive the comparison:

$$i \le n \implies f_i(n) < R_{35}(\vec{f}, n),$$
$$p \in \mathcal{P}_n \implies \mathsf{metavalue}(p) < R_{35}(\vec{f}, n).$$

The first is exactly the progressiveness property required by BusyLean's W-tree-ranked lift theorem.

7.3 The seed, schedule, and rank

Define the previous construction as a family rather than importing its final constant:

$$\mathsf{family}_{34}(n) = F_{34}(F_{34}(n)).$$

Then, definitionally,

$$\mathsf{family}_{34}(42) = C_{34}.$$

The new schedule and rank are autonomous W-trees obtained by applying `Autonomous.hyper` $\mathsf{family}_{34}(n)+42$ times above the previous schedule and then above the new schedule. This preserves the architecture of the accepted contender while enlarging both control trees by the full previous family.

Finally,

$$F_{35}(n) = \mathsf{wDiagonalLift}(\mathsf{schedule}_{35}(n))(\mathsf{rank}_{35}(n))(\mathsf{metaTransformer})(\mathsf{family}_{34})\ n\ n,$$

and

$$\boxed{C_{35} = F_{35}(F_{35}(42)).}$$

8 Formal comparison with Contender 34

8.1 The proof chain

The comparison file proves the following sequence.

Theorem	Statement and role
`lt_reflectMetaFamily`	Any number below the defining maximum is strictly below `reflectMetaFamily`.
`oracle_lt_reflectMetaFamily`	Every visible seed coordinate is strictly reflected.
`metavalue_lt_reflectMetaFamily`	Every enumerated program value is strictly reflected.
`metaTransformer_progressive`	The transformer satisfies the hypothesis of the W-ranked lift theorem.
`family34_lt_F35`	For every n, $\mathsf{family}_{34}(n) < F_{35}(n)$.
`raw_meta_reflection_lt_F35`	One raw reflection step is strictly below the lifted hierarchy.
`metavalue_lt_F35`	Every enumerated program value under the packed seed is below $F_{35}(n)$.
`input_lt_family34`	$n < \mathsf{family}_{34}(n)$.

Theorem	Statement and role
`input_lt_F35`	$n < F_{35}(n)$.
`contender_34_lt_contender_35`	The required final comparison.

The final proof is only two strict transitivity steps:

```
theorem contender_34_lt_contender_35 :
    contender_34 < contender_35 :=
  Nat.lt_trans (family34_lt_F35 42)
    (input_lt_F35 (F35 42))
```

The first term has left side $\mathsf{family}_{34}(42) = C_{34}$ by definitional reduction. Its right side is $F_{35}(42)$. The second term places $F_{35}(42)$ below $F_{35}(F_{35}(42)) = C_{35}$.

8.2 Why this is not a disguised use of the previous constant

The new definition imports `F34`, `schedule34`, and the reusable constructional infrastructure. It never mentions `contender_34`. The audit computes the transitive constant dependency closure of `contender_35` and rejects the build if any contender constant from 5 through 34 occurs.

This distinction matters. Writing $C_{35} = C_{34} + 1$ would be formally larger but mathematically vacuous. Writing $\mathsf{family}_{34}(n) = F_{34}(F_{34}(n))$ exposes the previous algorithm as an oracle family to a stronger closure construction, which is the intended BusyLean pattern.

9 Constructive computability

9.1 Why the number exists as a computation

Every component of the definition is constructive:

1. raw terms up to a given depth form a finite recursively generated list;
2. type inference is a terminating recursive function with decidable type equality;
3. typed metalanguage evaluation is structural recursion;
4. `metaLangLift` forms an inductive tree without enumerating all branches during elaboration;
5. `metaBoostT` is structural recursion on a countable tree;
6. addresses have a decidable well-founded descent relation;
7. the existing System-F normalizer is total on terms equipped with its formal strong-normalization witness;
8. finite maxima, family packing, autonomous W-tree realization, and the W-ranked lift are executable Lean definitions;
9. the outer expression applies total functions to the numeral 42.

Thus `contender_35 : Nat` is computable in the ordinary recursion-theoretic sense: a sufficiently idealized machine following Lean's evaluator eventually returns a unary/binary representation of a natural number. Feasibility is not claimed. As with prior BusyLean contenders, actually requesting the numeral is prohibitive and unnecessary.

9.2 No hidden nonconstructive shortcut

The metalanguage module contains no `noncomputable`, `partial`, `unsafe`, `opaque`, `extern`, or custom axiom declaration. Its semantic function spaces are constructive Lean function types. The proof layer may inherit Lean's standard propositional extensionality and quotient soundness through earlier modules, exactly within the repository's stated allowance; the numerical definition itself does not become an oracle thereby.

10 Why the increase is significant

10.1 Parameter growth versus type growth

There are two fundamentally different ways to enlarge a hierarchy.

Parameter growth. Keep the same closure operator and use a larger finite count, ordinal index, schedule, rank, or starting argument. This can be enormous, but the class of one-step closure rules is unchanged.

Type growth. Promote the closure operator to an argument available inside the language, then close under programs that manipulate it. New one-step rules become definable because programs can synthesize a transformer and ask the previous closure mechanism to close over it.

Contender 35 performs both, but the second is decisive. It changes $\mathsf{Lang} \in \mathcal{M}$ from fixed metalogical infrastructure into an object-language capability and then iterates the resulting operation $\mathsf{MetaLang} : \mathcal{M} \to \mathcal{M}$ transfinitely.

10.2 A schematic separation example

At a first-order stage, a program may use a distinguished transformer b, compose it, and recurse with it. Which transformer is promoted by the next Lang step is fixed externally.

At a metalanguage stage, a program can compute a transformer

$$q_x(t) = \mathsf{treeRec}(\ldots, x, t)$$

depending on run-time data x, then form $L(q_x)(u)$ inside the same program. It may repeat this under a transfinite tree recursion whose branches select different q_x. The set of promoted languages is therefore itself program-generated.

This is the operational content of the type jump. It is stronger evidence than a decorative increase in the numeral 42 or one extra use of the existing lift.

10.3 Four amplifications are composed

The final number combines four distinct amplifiers:

1. all closed metalanguage programs at all finite sizes;
2. transfinite iteration of metalanguage closure to height $T_{34}(n)$;
3. reflection over all bounded System-F ladder programs whose atoms are addresses of the resulting tree;
4. W-tree-ranked iteration over schedules and ranks expanded using the entire previous family.

Any one of these would yield a larger construction. Their composition is why Contender 35 is intended as a serious successor rather than an incremental patch.

10.4 What is and is not proved

The Lean theorem proves the exact natural-number inequality $C_{34} < C_{35}$. The source also proves uniform domination of $\mathsf{family}_{34}(n)$ by $F_{35}(n)$, plus bounds for every enumerated metalanguage-tree program.

It does *not* prove an exact fast-growing-hierarchy index or a strict ordinal inequality between the semantic ranks of $T_{34}(n)$ and $T_{35}(n)$. Assigning proof-theoretic ordinals to languages with these higher-order semantic closure operators would itself be a substantial research project. The report therefore describes the growth-rate jump as a structural and type-theoretic argument, not as a completed ordinal analysis.

11 Safety, predicativity, and consistency posture

11.1 No self-interpreter

A tempting but dangerous route would be to encode the metalanguage's own syntax and evaluator inside itself, then diagonalize. Such a construction would need a delicate normalization or consistency proof and could accidentally introduce a partial evaluator. The present proposal does not do this.

The semantic parameter `liftF` is supplied from Lean's metalanguage. A program can call it only through a typed primitive. It cannot inspect its code, modify its evaluator, or form an untyped self-application.

11.2 Predicative simple types

All types are generated from `nat` and `tree` by the arrow constructor. There is no universe object containing all simple types and no quantification over the totality to which the current type belongs. Higher functionals arise from finite arrow nesting, and transfinite iteration is over an inductive tree. This keeps the new module within the same conservative semantic style as the accepted tree-language modules.

11.3 Fallback constructors

The repository's `CTree` has constructors `supL` and `supH` used by earlier collapsing machinery. The previous `boostT` maps these to the base transformer, rather than attempting a new recursion principle over their payloads. `metaBoostT` follows the same convention, returning `langLift`. This makes the extension compatible with the established structural-recursion boundary.

12 Verification and audit protocol

12.1 Exact environment

The package targets the repository's pinned toolchain, Lean 4.34.0, and its existing Lake dependency graph. Verification is designed to run after the accepted Contender 34 modules are precompiled, matching the contest's rule that historical contenders may be cached.

12.2 Checks performed

The isolated audit performs four classes of checks:

1. kernel elaboration of `TreeLanguage4`, `Contender35`, and `Comparison35`;
2. `#print axioms` on the evaluator, the closure operators, the new number, the comparison theorem, and the program-value bound;
3. transitive dependency-closure rejection of all constants `contender_5` through `contender_34`;
4. timing of the three new module builds without evaluating the number.

12.3 Verification status

The exact Lean 4.34.0 repository build succeeded at commit `f08df5f2a4cb` in GitHub Actions run `38440600428`; the verification branch's full name and complete commit hash are recorded in `VERIFICATION.md`. With the accepted baseline precompiled, the measured module times were 0.53 seconds for `TreeLanguage4`, 0.33 seconds for `Contender35`, and 0.33 seconds for `Comparison35`. The axiom report is exactly within the repository allowance: the evaluator uses no axioms; the tree closures use `propext`; the number and comparison theorem use `propext` and `Quot.sound`. The dependency closure contained 5,534 constants and none of the thirty forbidden contender constants 5 through 34.

The archive includes `VERIFICATION.md` and machine-readable/log artifacts recording the exact commit, workflow run, diagnostics, timings, and axiom output. They provide the complete reproducible record of the successful check.

13 Integration into BusyLean

The proposal is deliberately modular. Integration requires adding the following imports after Contender 34:

```
import BusyLean.Ordinal.TreeLanguage4
import BusyLean.Contender35
import BusyLean.Comparison35
```

The supplied integration script copies the three modules and the audit into an existing checkout. The main repository audit should then add `#print axioms` commands for the new declarations and an `#assert_not_in_closure` block for `contender_35`. The check script's `PREVIOUS` module should move from `Comparison33` or `Comparison34`, as appropriate to its current organization, to `Comparison34`, while the timed definition/proof targets become the new modules.

No change to earlier accepted definitions is required. The proposal can be reviewed as a three-module extension.

14 Alternative designs considered

14.1 Another tree-ordinal stacking round

One could define a larger height such as $\mathsf{boostT}(T_{34})(T_{34})$ and otherwise copy Contender 34. This is constructive and easy to compare, but remains within the same first-order operator Lang. It would be a parameter increase, not a new closure principle.

14.2 A second W-ranked lift only

One could nest `wDiagonalLift` around F_{34} with larger schedules and ranks. This would certainly dominate the seed, but would leave the reflected object language unchanged. The present proposal retains the stronger schedule and rank while also changing the language layer.

14.3 Untyped universal interpreters

Enumerating arbitrary Lean programs or Turing machines would permit a dramatic diagonal, but totality would become undecidable. Bounding execution by fuel would restore computability at the cost of turning the construction into a resource-bound busy-beaver surrogate, contrary to the repository's proof- theoretic style. The typed semantic language gives a totality certificate by construction.

14.4 Impredicative language universes

A universe whose elements include its own closure operator could promise still larger strength. It would require a new reducibility or model construction and would risk a Girard-style circularity. A one-level predicative metalanguage is a safer and already substantial advance.

15 Further research program

15.1 Formal embedding and rank theorems

The most immediate strengthening is to formalize the embedding of the previous `TL` language into `MTL`. Desirable theorems include:

$$\begin{aligned} \mathsf{eval}_{\mathrm{MTL}}(\iota p, L, b) &= \mathsf{eval}_{\mathrm{TL}}(p, b), \\ \mathsf{programTree}(b, m) &\preccurlyeq \mathsf{metaLangLift}(L, b, t) \quad \text{for an explicit branch embedding.} \end{aligned}$$

One can then seek address embeddings between $T_{34}(n)$ and $T_{35}(n)$, not merely inclusion as branch zero.

15.2 All finite meta-levels

Define recursively

$$\mathcal{O}_0 = T \to T, \qquad \mathcal{O}_{k+1} = \mathcal{O}_k \to \mathcal{O}_k.$$

Contender 35 uses level one. A family of intrinsically typed languages could make k an input and close over operators at every finite level. The main technical issue is representing the denotation of $\mathcal{O}_k$ without making Lean elaborate infeasibly large dependent terms.

15.3 Tree-indexed functional levels

A more ambitious extension would replace the natural level k by a tree ordinal and define a dependent family of functional types with products at limits, paralleling Contender 31's transfinite closure types. The language would then manipulate closure operators whose *types* are indexed by the same tree used for iteration.

15.4 A typed code universe with certified evaluation

Instead of external finite enumeration, one could add a predicative code type for metalanguage terms and a total evaluator whose termination is proved by a logical relation. Programs could construct certified codes dynamically. The challenge is avoiding a self-code that invalidates stratification.

15.5 Exact proof-theoretic calibration

The hierarchy appears related to autonomous progressions of reflection principles and higher-type recursion over well-founded trees. A serious calibration would require:

1. a theory whose provably total functionals match the metalanguage stages;
2. a notation/collapse system for its reflection closure;
3. upper and lower simulations between that theory and the Lean definitions;
4. a comparison with the W-ranked numerical amplifier.

Until such work is done, exact ordinal labels should be treated as conjectural.

15.6 Performance engineering

The raw enumerator intentionally repeats many terms. A semantics-preserving quotient by type, size, and alpha-equivalence could greatly reduce small smoke tests. More importantly, the proof modules should remain insensitive to the enumeration's actual size: no command should force evaluation beyond tiny examples. Automated checks can inspect reducibility traces and generated code to guard against accidental evaluation.

16 Conclusion

Contender 35 is designed as the next conceptual step after languages stacked along tree ordinals. It promotes the language-closure operator to a typed primitive, closes under programs that manipulate that operator, and iterates the resulting metalanguages along Contender 34's own tree. The construction is predicative, total by structural recursion, and compatible with BusyLean's existing normalization and W-tree reflection infrastructure.

The exact theorem $C_{34} < C_{35}$ follows from uniform family domination and strict inflationarity. The more ambitious claim—that the number is not just larger but belongs to a substantially stronger growth regime—is supported by the one-functional-type jump and the transfinite closure architecture. The package avoids pretending that this structural evidence is already an exact ordinal analysis.

A Complete Lean source: the metalanguage

```
import BusyLean.Ordinal.TreeLanguage3

/-!
# A metalanguage for tree-ordinal languages

`TreeLanguage3` iterates the first-order language-closure operator `langLift`
along a countable tree ordinal.  This module raises the construction by one
functional type.  Its object language has two primitives:
```

```

* `boost : tree -> tree`, an arbitrary tree transformer;
* `lift : (tree -> tree) -> tree -> tree`, an arbitrary language-closure
  operator acting on tree transformers.

The language can therefore construct a transformer, pass it to the lower-level
language-closure operator, and continue computing with the resulting tree.
Closed programs of type `tree -> tree` are enumerated semantically, exactly as
in `TreeLanguage2`.

`metaLangLift L b` is the whole metalanguage over closure operator `L` and base
transformer `b`.  `metaBoostT` then iterates this operation along a countable
tree ordinal.  At input `n`, the height, base-transformer index, and argument
are all built from Contender 34's tree.
-/

namespace BusyLean.Ordinal

namespace MTL

/-- Intrinsically typed metalanguage terms.  The type system is the same
simply typed system used by `TL`; only the primitive operations are richer. -/
inductive Tm : List TL.Ty → TL.Ty → Type where
  | var : TL.Var Γ a → Tm Γ a
  | lam : Tm (a :: Γ) b → Tm Γ (.arr a b)
  | app : Tm Γ (.arr a b) → Tm Γ a → Tm Γ b
  | zero : Tm Γ .nat
  | succ : Tm Γ .nat → Tm Γ .nat
  | natRec : Tm Γ a → Tm Γ (.arr a a) → Tm Γ .nat → Tm Γ a
  | tzero : Tm Γ .tree
  | tsucc : Tm Γ .tree → Tm Γ .tree
  | tlim : Tm Γ (.arr .nat .tree) → Tm Γ .tree
  | treeRec : Tm Γ a → Tm Γ (.arr a a) →
      Tm Γ (.arr (.arr .nat a) a) → Tm Γ .tree → Tm Γ a
  | boost : Tm Γ .tree → Tm Γ .tree
  | lift : Tm Γ (.arr .tree .tree) → Tm Γ .tree → Tm Γ .tree

/-- Semantic interpreter.  It is total by structural recursion on the typed
term; no normalization theorem and no classical principle are needed. -/
def Tm.eval
    (liftF : (CTree → CTree) → CTree → CTree)
    (boostF : CTree → CTree) :
    Tm Γ a → TL.Env Γ → TL.Ty.den a
  | .var v, ρ => TL.Var.get v ρ
  | .lam t, ρ => fun x => t.eval liftF boostF (x, ρ)
  | .app t u, ρ => (t.eval liftF boostF ρ) (u.eval liftF boostF ρ)
  | .zero, _ => (0 : Nat)
  | .succ t, ρ => Nat.succ (t.eval liftF boostF ρ)
  | .natRec z s n, ρ =>
      TL.natIter (z.eval liftF boostF ρ) (s.eval liftF boostF ρ)
        (n.eval liftF boostF ρ)
  | .tzero, _ => (Tre.zero : CTree)
  | .tsucc t, ρ => (Tre.succ (t.eval liftF boostF ρ) : CTree)
  | .tlim f, ρ => (Tre.lim (f.eval liftF boostF ρ) : CTree)
  | .treeRec z s l t, ρ =>
      TL.treeIter (z.eval liftF boostF ρ) (s.eval liftF boostF ρ)
        (l.eval liftF boostF ρ) (t.eval liftF boostF ρ)
  | .boost t, ρ => boostF (t.eval liftF boostF ρ)
  | .lift b t, ρ =>
```

```
      liftF (b.eval liftF boostF ρ) (t.eval liftF boostF ρ)

/-! ## Raw terms and type checking -/

/-- Raw syntax; lambdas carry their argument type. -/
inductive Raw where
  | var : Nat → Raw
  | lam : TL.Ty → Raw → Raw
  | app : Raw → Raw → Raw
  | zero : Raw
  | succ : Raw → Raw
  | natRec : Raw → Raw → Raw → Raw
  | tzero : Raw
  | tsucc : Raw → Raw
  | tlim : Raw → Raw
  | treeRec : Raw → Raw → Raw → Raw → Raw
  | boost : Raw → Raw
  | lift : Raw → Raw → Raw

/-- Cast an intrinsically typed term after decidable type equality. -/
def cast? {Γ : List TL.Ty} {a : TL.Ty} (b : TL.Ty)
    (t : Tm Γ a) : Option (Tm Γ b) :=
  if h : a = b then some (h ▸ t) else none

/-- Sound-by-construction type inference from raw terms. -/
def infer (Γ : List TL.Ty) : Raw → Option (Σ a, Tm Γ a)
  | .var n => (TL.lookup Γ n).map fun ⟨a, v⟩ => ⟨a, .var v⟩
  | .lam a r =>
      (infer (a :: Γ) r).map fun ⟨b, t⟩ => ⟨.arr a b, .lam t⟩
  | .app r s => do
      let ⟨ab, t⟩ ← infer Γ r
      let ⟨_, u⟩ ← infer Γ s
      match ab, t with
      | .arr a b, t => do
          let u' ← cast? a u
          pure ⟨b, .app t u'⟩
      | _, _ => none
  | .zero => some ⟨.nat, .zero⟩
  | .succ r => do
      let ⟨_, t⟩ ← infer Γ r
      let t' ← cast? .nat t
      pure ⟨.nat, .succ t'⟩
  | .natRec z s n => do
      let ⟨a, tz⟩ ← infer Γ z
      let ⟨_, ts⟩ ← infer Γ s
      let ⟨_, tn⟩ ← infer Γ n
      let ts' ← cast? (.arr a a) ts
      let tn' ← cast? .nat tn
      pure ⟨a, .natRec tz ts' tn'⟩
  | .tzero => some ⟨.tree, .tzero⟩
  | .tsucc r => do
      let ⟨_, t⟩ ← infer Γ r
      let t' ← cast? .tree t
      pure ⟨.tree, .tsucc t'⟩
  | .tlim r => do
      let ⟨_, t⟩ ← infer Γ r
      let t' ← cast? (.arr .nat .tree) t
      pure ⟨.tree, .tlim t'⟩
  | .treeRec z s l r => do
```

```
      let ⟨a, tz⟩ ← infer Γ z
      let ⟨_, ts⟩ ← infer Γ s
      let ⟨_, tl⟩ ← infer Γ l
      let ⟨_, tr⟩ ← infer Γ r
      let ts' ← cast? (.arr a a) ts
      let tl' ← cast? (.arr (.arr .nat a) a) tl
      let tr' ← cast? .tree tr
      pure ⟨a, .treeRec tz ts' tl' tr'⟩
  | .boost r => do
      let ⟨_, t⟩ ← infer Γ r
      let t' ← cast? .tree t
      pure ⟨.tree, .boost t'⟩
  | .lift b r => do
      let ⟨_, tb⟩ ← infer Γ b
      let ⟨_, tr⟩ ← infer Γ r
      let tb' ← cast? (.arr .tree .tree) tb
      let tr' ← cast? .tree tr
      pure ⟨.tree, .lift tb' tr'⟩

/-! ## Enumeration -/

/-- Raw metalanguage terms of size at most `n`, with variables below `n`. -/
def rawsUpTo : Nat → List Raw
  | 0 => []
  | n + 1 =>
      let rs := rawsUpTo n
      let ts := TL.tysUpTo n
      (List.range n).map .var ++ [.zero, .tzero] ++
      ts.flatMap (fun a => rs.map (.lam a)) ++
      rs.flatMap (fun r => rs.map (.app r)) ++
      rs.map .succ ++ rs.map .tsucc ++ rs.map .tlim ++ rs.map .boost ++
      rs.flatMap (fun b => rs.map fun r => .lift b r) ++
      rs.flatMap (fun z => rs.flatMap fun s => rs.map (.natRec z s)) ++
      rs.flatMap (fun z =>
        rs.flatMap fun s => rs.flatMap fun l => rs.map (.treeRec z s l))

/-- Closed metalanguage programs of type `tree -> tree` and size at most `m`,
interpreted as actual total Lean functions. -/
def funsUpTo
    (liftF : (CTree → CTree) → CTree → CTree)
    (boostF : CTree → CTree) (m : Nat) : List (CTree → CTree) :=
  (rawsUpTo m).filterMap fun r =>
    match infer [] r with
    | some ⟨.arr .tree .tree, t⟩ => some (t.eval liftF boostF ())
    | _ => none

end MTL

/-- The whole metalanguage over closure operator `liftF` and tree transformer
`b`, applied to `t`.  Every closed `tree -> tree` program of every finite size
occurs in the two nested limits. -/
def metaLangLift
    (liftF : (CTree → CTree) → CTree → CTree)
    (b : CTree → CTree) (t : CTree) : CTree :=
  .lim fun m => .lim fun i =>
    (((MTL.funsUpTo liftF b m)[i]?).map fun p => p t).getD .zero

/-- Metalanguages stacked along a countable tree ordinal.  The zero stage uses
`langLift`; a successor closes a language that can invoke the entire previous
```

```
/-- closure operator; limits are pointwise limits. -/
def metaBoostT : CTree → (CTree → CTree) → CTree → CTree
  | .zero => langLift
  | .succ a => metaLangLift (metaBoostT a)
  | .lim f => fun b t => .lim fun i => metaBoostT (f i) b t
  | .supL _ => langLift
  | .supH _ => langLift

/-- The input tree for Contender 35.  Branch zero retains Contender 34's tree.
Every other branch iterates the metalanguage hierarchy to that same height,
starting from a first-order language hierarchy also indexed by Contender 34's
tree, and applies it to Contender 34's tree. -/
def metaLanguageTree (n : Nat) : CTree :=
  .lim fun i =>
    if i = 0 then treeStackedTree n
    else metaBoostT (treeStackedTree n)
      (boostT (treeStackedTree n)) (treeStackedTree n)

end BusyLean.Ordinal
```

B Complete Lean source: the number

```
import BusyLean.Ordinal.TreeLanguage4
import BusyLean.Contender34

/-!
# Contender 35: metalanguages stacked along tree ordinals

```
contender_35 := F35 (F35 42)
F35 n        := wDiagonalLift (schedule35 n) (rank35 n)
                  metaTransformer (fun _ => family34) n n
```

Contender 34 iterated a first-order language-closure operator along a tree
ordinal. Contender 35 raises the closure operator itself to an object-language
primitive and iterates the resulting metalanguages along Contender 34's tree.
The numerical reflection layer then runs over addresses of the resulting tree
and is seeded by the whole Contender 34 family.

Do not evaluate `contender_35`.
-/

namespace BusyLean.FL

open Ordinal

/-- Every accepted System-F term over addresses of the metalanguage tree is
strongly normalizing. -/
theorem sn_erase_metatree {f : Nat → Nat} (L : Nat) {fuel : Nat}
    {t : CTm (Addr (metaLanguageTree L))}
    (h : infer fuel [] [] t = some Ty.nat) : F.SN f (erase t) :=
  sn_of_typed (wfUniv f (addr_wf (metaLanguageTree L)))
    (infer_sound fuel t (goodCtx_nil []) h).1

/-- Value of a checked program over addresses of `metaLanguageTree L`, relative
```

```
35 to oracle `f`. -/
36 def metavalue (L : Nat) (f : Nat → Nat) (fuel : Nat)
37     (t : CTm (Addr (metaLanguageTree L))) : Nat :=
38   match h : infer fuel [] [] t with
39   | some A =>
40       if hA : A = Ty.nat then
41         F.decode
42           (F.nf f ⟨erase t, sn_erase_metatree L (h.trans (congrArg some hA))⟩)
43       else 0
44   | none => 0
45
46 /-- Programs of depth at most `n` over addresses of `metaLanguageTree n`. -/
47 def metatreeLadderUpTo (n : Nat) :
48     List (CTm (Addr (metaLanguageTree n))) :=
49   ctmsUpTo (addrUpTo (metaLanguageTree n) n) n
50
51 end BusyLean.FL
52
53 namespace BusyLean
54
55 open BigNumber (WTree)
56 open BusyLean.W
57
58 /-- Family-aware reflection over the metalanguage tree. -/
59 def reflectMetaFamily (fs : Family) (n : Nat) : Nat :=
60   n + 1 + BigNumber.maxN (prefixMax (fun i => fs i n) n)
61     (BigNumber.maximum (FL.metavalue n (packFamily fs) n)
62       (FL.metatreeLadderUpTo n))
63
64 def metaTransformer : Transformer := fun fs _ n => reflectMetaFamily fs n
65
66 /-- Contender 34's construction as a family.  In particular,
67 `family34 42 = contender_34`, definitionally. -/
68 def family34 (n : Nat) : Nat := F34 (F34 n)
69
70 /-- A schedule strictly above the previous schedule, expanded by the whole
71 Contender 34 family at the current input. -/
72 def schedule35 (n : Nat) : WTree Nat :=
73   Autonomous.realize
74     (Autonomous.hyper (family34 n + 42) (Autonomous.encode (schedule34 n)))
75
76 /-- A W-tree rank above the new schedule. -/
77 def rank35 (n : Nat) : WTree Nat :=
78   Autonomous.realize
79     (Autonomous.hyper (family34 n + 42) (Autonomous.encode (schedule35 n)))
80
81 /-- W-ranked hyperreflection over metalanguage address ladders, seeded with
82 Contender 34's family. -/
83 def F35 (n : Nat) : Nat :=
84   wDiagonalLift (schedule35 n) (rank35 n) metaTransformer
85     (fun _ => family34) n n
86
87 /-- **Contender 35.**  Never evaluate it. -/
88 def contender_35 : Nat := F35 (F35 42)
89
90 end BusyLean
```

C Complete Lean source: comparison theorems

```
import BusyLean.Contender35
import BusyLean.Comparison34

/-! # Contender 35: comparison theorems -/

namespace BusyLean

open BusyLean.W

/-- Any number bounded by the reflected maximum is strictly below the result. -/
theorem lt_reflectMetaFamily {fs : Family} {n m : Nat}
    (h : m ≤ BigNumber.maxN (prefixMax (fun i => fs i n) n)
      (BigNumber.maximum (FL.metavalue n (packFamily fs) n)
        (FL.metatreeLadderUpTo n))) :
    m < reflectMetaFamily fs n := by
  show m < n + 1 + _
  have hM : ∀ M : Nat, M + 1 ≤ n + 1 + M := fun M => by
    rw [Nat.add_comm M 1]
    exact Nat.add_le_add_right (Nat.le_add_left 1 n) M
  exact Nat.lt_of_le_of_lt h
    (Nat.lt_of_lt_of_le (Nat.lt_succ_self _) (hM _))

/-- Every visible coordinate of the oracle family is strictly reflected. -/
theorem oracle_lt_reflectMetaFamily (fs : Family) {i n : Nat}
    (hi : i ≤ n) : fs i n < reflectMetaFamily fs n :=
  lt_reflectMetaFamily
    (Nat.le_trans (le_prefixMax (fun j => fs j n) hi)
      (BigNumber.le_maxN_left _ _))

/-- Every enumerated program value is strictly reflected. -/
theorem metavalue_lt_reflectMetaFamily (fs : Family) {n : Nat}
    {t : FL.CTm (Ordinal.Addr (Ordinal.metaLanguageTree n))}
    (ht : t ∈ FL.metatreeLadderUpTo n) :
    FL.metavalue n (packFamily fs) n t < reflectMetaFamily fs n :=
  lt_reflectMetaFamily
    (Nat.le_trans
      (le_maximum (FL.metavalue n (packFamily fs) n) ht)
      (BigNumber.le_maxN_right _ _))

/-- The metalanguage reflection transformer is progressive. -/
theorem metaTransformer_progressive : Progressive metaTransformer :=
  fun seed _ _ _ hi => oracle_lt_reflectMetaFamily seed hi

/-- At every input, `F35` exceeds Contender 34's full family. -/
theorem family34_lt_F35 (n : Nat) : family34 n < F35 n :=
  ((wDiagonalLift_preservesProgressive (schedule35 n) (rank35 n))
    metaTransformer metaTransformer_progressive)
    (fun _ => family34) 0 n n (Nat.zero_le n)

/-- One raw metalanguage-reflection step is below the W-ranked lift. -/
theorem raw_meta_reflection_lt_F35 (n : Nat) :
    reflectMetaFamily (fun _ => family34) n < F35 n :=
  (wDiagonalLift_strictlyInflationary (schedule35 n) (rank35 n))
    metaTransformer metaTransformer_progressive
      (fun _ => family34) n n n (Nat.le_refl n)

```

```lean
/-- Every enumerated metalanguage-tree program, run with the packed seed, is
below `F35 n`. -/
theorem metavalue_lt_F35 {n : Nat}
    {t : FL.CTm (Ordinal.Addr (Ordinal.metaLanguageTree n))}
    (ht : t ∈ FL.metatreeLadderUpTo n) :
    FL.metavalue n (packFamily (fun _ => family34)) n t < F35 n :=
  Nat.lt_trans (metavalue_lt_reflectMetaFamily (fun _ => family34) ht)
    (raw_meta_reflection_lt_F35 n)

/-- The input is below the seed family. -/
theorem input_lt_family34 (n : Nat) : n < family34 n :=
  Nat.lt_trans (input_lt_F34 n) (input_lt_F34 (F34 n))

/-- `F35` is strictly inflationary on natural inputs. -/
theorem input_lt_F35 (n : Nat) : n < F35 n :=
  Nat.lt_trans (input_lt_family34 n) (family34_lt_F35 n)

/-- **Required comparison:** Contender 34 is strictly below Contender 35. -/
theorem contender_34_lt_contender_35 : contender_34 < contender_35 :=
  Nat.lt_trans (family34_lt_F35 42) (input_lt_F35 (F35 42))

/-- A useful transitive comparison with the preceding accepted contender. -/
theorem contender_33_lt_contender_35 : contender_33 < contender_35 :=
  Nat.lt_trans contender_33_lt_contender_34 contender_34_lt_contender_35

end BusyLean
```

D Artifact map

Artifact	Purpose
`BusyLean/Ordinal/TreeLanguage4.lean`	Typed metalanguage, interpreter, checker, enumeration, closure, and transfinite iteration.
`BusyLean/Contender35.lean`	Address-ladder value function, family reflection, schedule, rank, F_{35}, and the closed number.
`BusyLean/Comparison35.lean`	Progressiveness, program bounds, uniform family domination, and final comparison.
`test/Contender35Audit.lean`	Axiom reports and forbidden-constant closure check.
`article/contender35_metalanguage.tex`	Source of this report.
`article/contender35_metalanguage.pdf`	Compiled report.
`VERIFICATION.md`	Exact kernel-check status, diagnostics, timings, and workflow identifiers.
`apply_to_checkout.sh`	Idempotent integration helper.
`MANIFEST.sha256`	Integrity hashes for packaged artifacts.

References

[1] Kurt Gödel. Über eine bisher noch nicht benützte Erweiterung des finiten Standpunktes. *Dialectica*, 12:280–287, 1958.

[2] William W. Tait. Intensional interpretations of functionals of finite type I. *Journal of Symbolic Logic*, 32(2):198–212, 1967.

[3] Jean-Yves Girard, Yves Lafont, and Paul Taylor. *Proofs and Types*. Cambridge University Press, 1989.

[4] Solomon Feferman. Transfinite recursive progressions of axiomatic theories. *Journal of Symbolic Logic*, 27(3):259–316, 1962.

[5] William A. Howard. Ordinal analysis of terms of finite type. *Journal of Symbolic Logic*, 45(3):493–504, 1980.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction—CADE 28*, pages 625–635, 2021.

[7] Vladimir Reshetnikov. *BusyLean: name the biggest number, in Lean*. Repository documentation and formal source, consulted 13 October 2026.